Name of the module: mat_by_vec
Location in GIT repository: vspa_common\vspa-lib

Type of code:
VCPU assembly function (1 function)

VCPU Function API:
"mat_by_vec" family:

```
extern void mat_by_vec_chfx_chfx_chfx (
       vspa_complex_fixed16          *py,
       vspa_complex_fixed16 const    *px,
       vspa_complex_fixed16 const    *pa,
       uint32_t                       offset,
       uint32_t                       L,
       uint32_t                       M
                    );
extern void mat_by_vec_chfx_chfx_chfl (
       vspa_complex_fixed16          *py,
       vspa_complex_fixed16 const    *px,
       vspa_complex_float16 const    *pa,
       uint32_t                       offset,
       uint32_t                       L,
       uint32_t                       M
                    );
extern void mat_by_vec_chfx_chfx_rhfx (
       vspa_complex_fixed16          *py,
       vspa_complex_fixed16 const    *px,
       fixed16_t            const    *pa,
       uint32_t                       offset,
       uint32_t                       L,
       uint32_t                       M
                    );

extern void mat_by_vec_chfx_chfx_rhfl (
       vspa_complex_fixed16          *py,
       vspa_complex_fixed16 const    *px,
       float16_t            const    *pa,
       uint32_t                       offset,
       uint32_t                       L,
       uint32_t                       M
                    );
```

```
extern void mat_by_vec_chfx_chfx_rfl (
       vspa_complex_fixed16          *py,
       vspa_complex_fixed16 const    *px,
       float                const    *pa,
       uint32_t                       offset,
       uint32_t                       L,
       uint32_t                       M
                    );
extern void mat_by_vec_chfl_chfx_chfx (
       vspa_complex_float16          *py,
       vspa_complex_fixed16 const    *px,
       vspa_complex_fixed16 const    *pa,
       uint32_t                       offset,
       uint32_t                       L,
       uint32_t                       M
                    );
extern void mat_by_vec_chfl_chfx_chfl (
       vspa_complex_float16          *py,
       vspa_complex_fixed16 const    *px,
       vspa_complex_float16 const    *pa,
       uint32_t                       offset,
       uint32_t                       L,
       uint32_t                       M
                    );
extern void mat_by_vec_chfl_chfx_rhfx (
       vspa_complex_float16          *py,
       vspa_complex_fixed16 const    *px,
       fixed16_t            const    *pa,
       uint32_t                       offset,
       uint32_t                       L,
       uint32_t                       M
                    );

extern void mat_by_vec_chfl_chfx_rhfl (
       vspa_complex_float16          *py,
       vspa_complex_fixed16 const    *px,
       float16_t            const    *pa,
       uint32_t                       offset,
       uint32_t                       L,
       uint32_t                       M
                    );

extern void mat_by_vec_chfl_chfx_rfl (
       vspa_complex_float16          *py,
       vspa_complex_fixed16 const    *px,
       float                const    *pa,
       uint32_t                       offset,
       uint32_t                       L,
       uint32_t                       M
                    );
```

API description:

- py — pointer to the output buffer $\boldsymbol{y}$ in VCPU (dmem aligned)
- px — pointer to the first input buffer $\boldsymbol{x}_1$ in VCPU (dmem aligned)
- pa — pointer to the input buffer $\boldsymbol{a}_{Mx1}$ in VCPU
- `offset` — integer number which defines the offset in words between $\boldsymbol{x}_1$ and $\boldsymbol{x}_2$, $\boldsymbol{x}_2$ and $\boldsymbol{x}_3$, … etc. This number should be integer multiple of DMEM lines, i.e., offset = {32, 64, …, i*32, …}, where i is an integer number
- L — integer number which defines the number of DMEM lines used to store the output buffer $\boldsymbol{y}_{Kx1}$. L= ceil(K/32)
- M — integer number which defines the number of entries in the input buffer $\boldsymbol{a}_{Mx1}$

Implementation Plan:

This function performs the multiplication between a matrix (**X**) of size K x M by a column vector (**a**) of size M.

$$\boldsymbol{y}_{Kx1} = \boldsymbol{X}_{KxM} * \boldsymbol{a}_{Mx1} = [\boldsymbol{x}_1\ \boldsymbol{x}_2\ \cdots\ \boldsymbol{x}_M] * \begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_M \end{bmatrix} = a_1\boldsymbol{x}_1 + a_2\boldsymbol{x}_2 + \cdots + a_M\boldsymbol{x}_M$$

DMEM requirements:

Variable	Minimum size (half-words)	Minimum alignment (half-words)	Allocated by
py	L*64	64	Caller
px	L*64	64	Caller
pa	M*2	2	Caller
offset	2	2	Caller
L	2	2	Caller
M	2	2	Caller

Test Plan:

- Compare the output buffer with a matlab testbench